A MINI PROJECT REPORT ON

"RULE-BASED BOOK RECOMMENDATION SYSTEM USING PYTHON"

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE

OF

THIRD YEAR ENGINEERING (COMPUTER ENGINEERING)

SUBMITTED BY

ROHIT KHENGARE	ROLL NO: **19**
PRATHMESH KULKARNI	ROLL NO: **23**
ROHIT RAJPUT	ROLL NO: **37**



DEPARTMENT OF COMPUTER ENGINEERING
SRES'SHREE RAMCHANDRA COLLEGE OF ENGINEERING
LONIKAND, WAGHOLI, PUNE 412216
SAVITRIBAI PHULE PUNE UNIVERSITY

2026 - 2027

CERTIFICATE

This is to certify that the mini project report entitled

"RULE-BASED BOOK RECOMMENDATION SYSTE M USING PYTHON"

Submitted by

ROHIT KHENGARE	ROLL NO: **19**
PRATHMESH KULKARNI	ROLL NO: **23**
ROHIT RAJPUT	ROLL NO: **37**

is a Bonafede student of this institute and the work has been carried out by them under the supervision of **Prof. Harshada Rane** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for **Third Year of Engineering** (Computer Engineering).

Prof. Harshada Rane
(Guide)

Prof.Dr. Neelam Kumar
(HOD, Computer Engineering)

Prof. Dr. S.D. Shinde
(Principal, Shree Ramchandra College of Engineering)

Place: Pune
Date:

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to our guide, Prof. Harshada Rane, for her valuable guidance, constant encouragement and support throughout the development of this mini project, “Rule-Based Book Recommendation System Using Python”. Her suggestions helped us to understand the concepts of knowledge representation, IF-THEN rules, forward chaining and backward chaining in a practical way.

We are thankful to Prof. Dr. Neelam Kumar, Head of the Department of Computer Engineering, and to Prof. Dr. S.D. Shinde, Principal of Shree Ramchandra College of Engineering, for providing the facilities and a supportive academic environment. We also thank all the faculty members, our friends and our families for their help and motivation.

TABLE OF CONTENTS

1. ABSTRACT 4
2. INTRODUCTION 4
3. PROBLEM STATEMENT AND OBJECTIVES 4
4. AI CONCEPTS USED 4
5. SYSTEM DESIGN 5
6. INPUT AND OUTPUT 6
7. APPLICATIONS AND LIMITATIONS 7
8. CONCLUSION 7
9. REFERENCES 7

1. ABSTRACT

The Rule-Based Book Recommendation System is a knowledge-based AI system developed in Python. It recommends books from a knowledge base of **56 popular books** in seven genres (Fantasy, Sci-Fi, Mystery, Self-Help, Technology, History and Fiction). The user selects a genre, a reading level (Beginner, Intermediate, Advanced) and a mood (Relaxed, Excited, Thoughtful, Motivated).

Nine IF-THEN rules turn the level and the mood into new facts by **forward chaining**, for example *IF level = Beginner THEN lvl = Easy*. The books whose genre, level and tone match these facts are displayed together with the rules that fired. The user can also select one book and the system uses **backward chaining** to prove or disprove the goal *recommend(book)* and shows the complete proof. The project runs from VS Code, opens a web page automatically and uses only the Python standard library.

2. INTRODUCTION

Artificial Intelligence (AI) is the branch of computer science that builds software able to perform tasks that normally need human intelligence, such as reasoning and decision making. An **expert system** is an AI program that stores the knowledge of an expert as facts and rules and uses an **inference engine** to reach conclusions. Early systems such as MYCIN (medical diagnosis) were built in this way.

Knowledge is represented as **production rules** of the form IF condition THEN conclusion. In propositional logic, the rule P → Q together with the fact P allows us to conclude Q (*Modus Ponens*). Rules can be applied in two directions: **forward chaining** starts from the known facts and derives new facts (data driven), while **backward chaining** starts from a goal and searches for the facts that prove it (goal driven).

A recommendation system suggests items to a user according to the user's needs. This project is knowledge-based: it needs no user history, so it has no cold-start problem, and every recommendation can be explained by the rules that produced it.

3. PROBLEM STATEMENT AND OBJECTIVES

Choosing a good book from thousands of titles is difficult, because the right book depends on the reader's taste, reading level and mood. The aim of this project is to build a simple rule-based AI system that accepts these preferences, applies IF-THEN rules and recommends suitable books with a clear explanation. It runs on any computer with Python 3 and needs no external library.

Objectives

- To understand knowledge representation using facts and IF-THEN production rules.
- To implement forward chaining to find all books that suit the genre, level and mood of the user.
- To implement backward chaining to check whether one chosen book fits the user, with a proof tree.
- To build a knowledge base of 56 popular books and show the output on a simple web page opened from VS Code.

4. AI CONCEPTS USED

Concept	Description
Knowledge base	Facts (56 books, each with genre, level and tone) and 9 IF-THEN rules.
Production rule	IF condition THEN conclusion, e.g. IF mood = Relaxed THEN tone = Light.
Modus Ponens	From the fact P and the rule P → Q, the system derives Q.
Forward chaining	Data driven. Starts from user facts and fires rules until no new fact appears (function recommend).
Backward chaining	Goal driven. Starts from recommend(book) and proves sub-goals (functions prove, backward).
Inference engine	The part of the program that applies the rules to the facts.
Explanation facility	Shows the rules fired and the proof tree, as in MYCIN.

Rule Base (knowledge base of IF-THEN rules)

Rule	IF (condition)	THEN (conclusion)
R1	level=Beginner	lvl=Easy
R2	level=Intermediate	lvl=Easy
R3	level=Intermediate	lvl=Medium
R4	level=Advanced	lvl=Medium
R5	level=Advanced	lvl=Hard
R6	mood=Relaxed	tone=Light
R7	mood=Excited	tone=Thrilling
R8	mood=Thoughtful	tone=Deep
R9	mood=Motivated	tone=Inspiring
R10	genre matches AND lvl is allowed AND tone is allowed	recommend the book
R11	no book satisfies R10	ignore the tone and use genre + level only

Worked example (forward chaining): the user selects genre = Sci-Fi, level = Intermediate and mood = Relaxed. The known facts are *level = Intermediate* and *mood = Relaxed*. Rule R2 gives *lvl = Easy*, rule R3 gives *lvl = Medium* and rule R6 gives *tone = Light*. Among the eight Sci-Fi books, only **The Martian** (Easy, Light) satisfies rule R10, so it is recommended.

Worked example (backward chaining): to verify *recommend(The Martian)* the system must prove three sub-goals. *genre = Sci-Fi* is a known fact. For *lvl = Easy*, rule R1 fails because level = Beginner is not a fact, but rule R2 succeeds. For *tone = Light*, rule R6 succeeds because mood = Relaxed is a fact. All sub-goals are proved, so the book is recommended.

5. SYSTEM DESIGN

The system has five parts: the user interface (a web page served by Python and opened in the browser), the forward chaining engine, the backward chaining engine, the knowledge base (9 rules and 56 books, eight per genre) and the output module that shows the books, the rules fired and the proof tree.

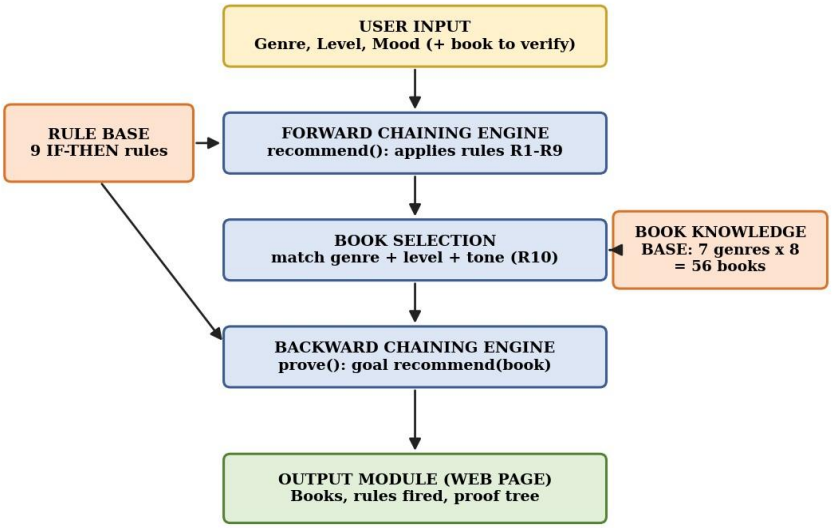


Figure 5.1: Architecture of the system

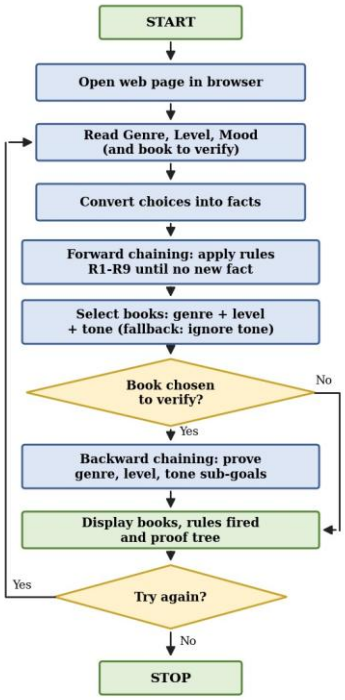


Figure 5.2: Flowchart

Algorithm

1. Start the program; the web page opens in the browser.
2. The user selects genre, level and mood (and optionally a book to verify).
3. Convert the choices into facts such as level = Beginner and mood = Relaxed.
4. Forward chaining: apply rules R1 to R9 repeatedly until no new fact can be derived.
5. Collect the allowed levels (lvl) and tones (tone) from the derived facts.
6. Select the books whose genre, level and tone match (rule R10). If none match, ignore the tone (rule R11).
7. Display the recommended books and the rules fired.
8. If a book was chosen, backward chaining: set the goal recommend(book), split it into the sub-goals genre, lvl and tone and prove each one using the facts and rules.
9. Display the proof tree and the result (RECOMMENDED or NOT recommended). Ask whether to try again; if yes go to step 2, else stop.

6. INPUT AND OUTPUT

The user gives three inputs: genre, level and mood. The system displays the recommended books and the rules that fired. The table shows the outputs for eight different inputs (* means rule R11 was used because no book matched the mood).

No.	Genre	Level	Mood	Rules fired	Books	Books recommended
1	Fantasy	Beginner	Relaxed	R1, R6	2	Percy Jackson: The Lightning Thief, The Chronicles of Narnia
2	Sci-Fi	Intermediate	Relaxed	R2, R3, R6	1	The Martian
3	Mystery	Beginner	Excited	R1, R7	3	A Study in Scarlet, And Then There Were None (+1 more)
4	Self-Help	Advanced	Motivated	R4, R5, R9	1	The 7 Habits of Highly Effective People
5	Technology	Advanced	Thoughtful	R4, R5, R8	5	Artificial Intelligence: A Modern Approach, Clean Code (+3 more)
6	History	Intermediate	Thoughtful	R2, R3, R8	3	Sapiens, Homo Deus (+1 more)
7	Fiction	Beginner	Thoughtful	R1, R8	3	The Fault in Our Stars, Me Before You (+1 more) *
8	Fantasy	Advanced	Relaxed	R4, R5, R6	4	The Lord of the Rings, A Game of Thrones (+2 more) *

Input and Output Screenshots

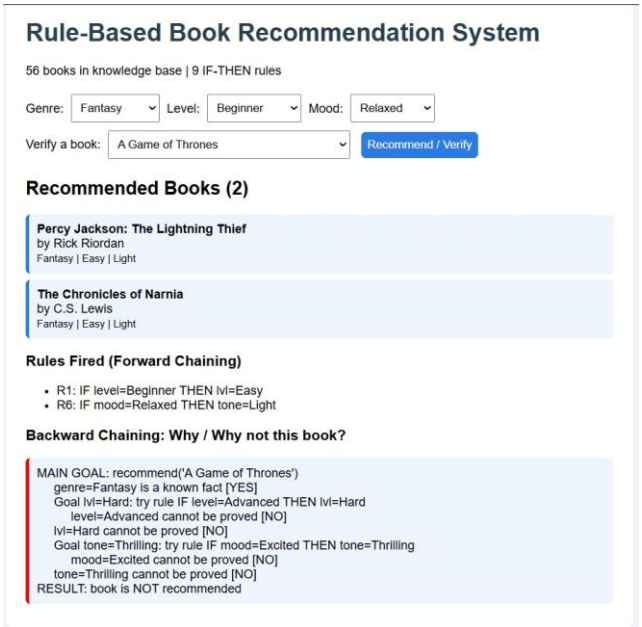


Figure 6.1: Run 1 (Fantasy, Beginner, Relaxed; verify A Game of Thrones)

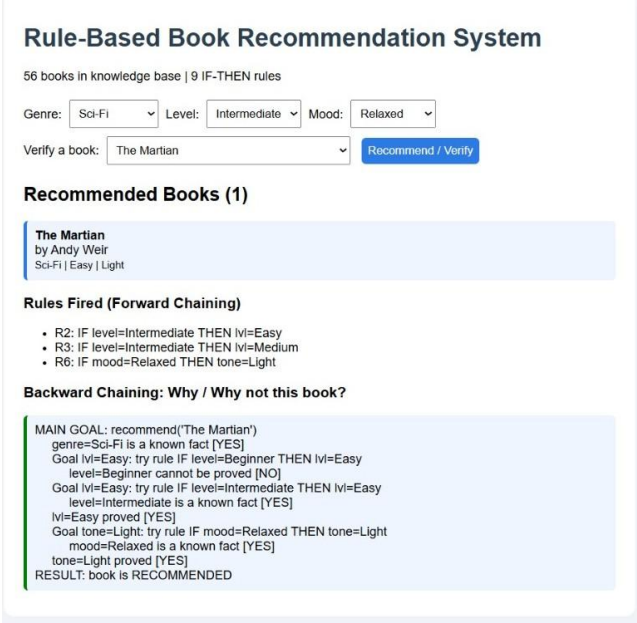


Figure 6.2: Run 2 (Sci-Fi, Intermediate, Relaxed; verify The Martian)

Run 1: Input: Fantasy, Beginner, Relaxed. Output: 2 books, Percy Jackson: The Lightning Thief and The Chronicles of Narnia (rules R1 and R6 fired). Backward chaining for *A Game of Thrones* fails because lvl = Hard and tone = Thrilling cannot be proved, so it is NOT recommended.

Run 2: Input: Sci-Fi, Intermediate, Relaxed. Output: 1 book, The Martian (rules R2, R3 and R6 fired). Backward chaining proves all three sub-goals, so the book is RECOMMENDED.

7. APPLICATIONS AND LIMITATIONS

Applications: book suggestion in school and college libraries, small online book stores and reading apps; teaching forward and backward chaining in AI laboratories; the same design can recommend movies, music or courses.

Limitations: the rules and the book data are written by the developer and are not learned from users; the knowledge base is small (7 genres, 56 books); the books are not ranked by rating or popularity; the system works with one genre at a time.

8. CONCLUSION

The system stores 56 books and 9 IF-THEN rules, uses forward chaining to find all suitable books and backward chaining to explain why a particular book is or is not suitable. Testing showed that the results are correct and easy to explain through the rules fired and the proof tree. The project can be extended with more books, ratings, more moods and rules learned from data.

9. REFERENCES

1. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, Pearson.
2. B. G. Buchanan and E. H. Shortliffe, *Rule-Based Expert Systems: The MYCIN Experiments*, Addison-Wesley, 1984.
3. Python Software Foundation, *http.server - HTTP servers*, docs.python.org/3/library/http.server.html.
4. Savitribai Phule Pune University, Third Year Computer Engineering (2024 Pattern), PCC301COM Artificial Intelligence.